The Arcade: Getting Started

Table of Contents

1. GitHub Account Creation . . . 2
2. Joining the GitHub Organization . . . 3
3. Team Divisions and Responsibilities . . . 3
   3.1. TheArcade Collaborator . . . 3
   3.2. Producer . . . 4
   3.3. Artist . . . 4
   3.4. Lead Artist . . . 4
   3.5. Developer . . . 5
   3.6. Lead Developer . . . 5
4. Installing GitHub Desktop . . . 6
5. Installing Godot 4.7.2 . . . 6
6. Cloning the Repository . . . 6
   6.1. Local and Remote Copies . . . 7
7. The Complete Contribution Workflow . . . 7
   7.1. 1. Discuss the Work . . . 8
   7.2. 2. Create the Issue . . . 9
      7.2.1. Title . . . 9
      7.2.2. Label . . . 9
      7.2.3. Objective . . . 9
      7.2.4. Description . . . 9
      7.2.5. Acceptance Criteria . . . 10
   7.3. 3. Wait for Lead Approval . . . 10
   7.4. 4. Wait for Producer Approval . . . 10
   7.5. 5. Add the Issue to the Product Backlog . . . 11
   7.6. 6. Sprint Start and Issue Assignment . . . 11
   7.7. 7. Fetch . . . 11
   7.8. 8. Pull . . . 12
   7.9. 9. Create the Branch . . . 12
   7.10. 10. Work on the Branch . . . 13
   7.11. 11. Review Changes Before Committing . . . 13
   7.12. 12. Commit . . . 13
   7.13. 13. Push . . . 14
   7.14. 14. Create the Pull Request . . . 14
   7.15. 15. Lead Review and Approval . . . 15
      7.15.1. If Changes Are Requested . . . 15
   7.16. 16. Producer Review and Approval . . . 16

7.17. 17. Producer Merge . . . 16
8. Workflow Summary . . . 16
9. Git and GitHub Vocabulary . . . 17
9.1. Repository . . . 17
9.2. Local Repository . . . 17
9.3. Remote Repository . . . 17
9.4. Issue . . . 17
9.5. Product Backlog . . . 17
9.6. Sprint . . . 17
9.7. Branch . . . 17
9.8. Fetch . . . 17
9.9. Pull . . . 17
9.10. Commit . . . 18
9.11. Push . . . 18
9.12. Pull Request . . . 18
9.13. Review . . . 18
9.14. Merge . . . 18
10. TheArcade Golden Rules . . . 18
11. What You Are Not Expected to Know Yet . . . 19
12. When to Ask for Help . . . 19
13. Quick Pre-Work Checklist . . . 20
14. Useful Official Guides . . . 20
15. Final Reminder . . . 20

Welcome to **The Arcade**.

This guide contains the minimum setup, team structure, and Git/GitHub workflow every collaborator should understand before contributing to the project.



If GitHub Desktop reports a merge conflict, unexpected branch state, failed operation, or anything you do not understand, stop and ask a Lead or Producer before attempting to fix it yourself.

1. GitHub Account Creation

Every collaborator needs a personal GitHub account. Your GitHub account is your identity inside the organization and is used to attribute Issues, commits, Pull Requests, reviews, and other project activity to you.

Use GitHub's official account-creation guide:

Creating an account on GitHub

Before continuing, make sure that:

- You preferably used your institutional email.
- Your email address is verified.
- You can sign in successfully at GitHub.
- Your username is appropriate for use within the team.
- You have access to the email address associated with the account, since organization invitations may be delivered there.

GitHub strongly recommends enabling two-factor authentication. You are encouraged to enable it to protect your account.

2. Joining the GitHub Organization

The Arcade uses a GitHub organization so that repositories, Issues, teams, permissions, and Pull Requests can be managed in one shared location.

A Producer, or other authorized organization member will send you an invitation. You must accept the invitation before you can access organization resources that are restricted to members.

After accepting the invitation:

- Confirm that TheArcade organization appears under your GitHub organizations.
- Confirm that you can view the project repository.
- Confirm that you have been placed in the appropriate GitHub team or teams.
- If you cannot access the repository, contact a Lead or Producer.

3. Team Divisions and Responsibilities

Team roles are **not generally mutually exclusive**. A collaborator may participate in more than one discipline when workload, interest, and project needs allow it.

Lead roles are mutually exclusive.

All specialized roles inherit the responsibilities of **The Arcade Collaborator**.

3.1. TheArcade Collaborator

Every organization member is first a Collaborator.

Responsibilities:

- Follow the team's Issue, branch, commit, Push, and Pull Request workflow.
- Communicate progress and blockers honestly.
- Work only on approved and assigned tasks unless otherwise instructed.
- Keep work within the scope of the assigned Issue.

- Participate in Sprint activities relevant to your work.
- Review the acceptance criteria before considering an Issue complete.
- Keep project files organized according to team conventions.
- Ask for help when an unfamiliar Git, Godot, or project-management situation appears.
- Treat reviews as part of collaboration rather than as personal evaluation.

3.2. Producer

The Producer coordinates the project workflow and acts as the final approval gate for work entering or leaving the active development process.

Responsibilities:

- Coordinate the Product Backlog and Sprint planning process.
- Confirm that proposed work fits current project priorities and scope.
- Approve Issues for addition to the Product Backlog after Lead review.
- Help decide which backlog items enter a Sprint.
- Coordinate Issue assignment at Sprint start.
- Track project risks, blockers, dependencies, and deadlines.
- Provide the final approval on Pull Requests after the appropriate Lead has reviewed them.
- Merge approved Pull Requests into the target branch when appropriate.
- Protect the stability of `main` and other shared integration branches.

3.3. Artist

Artists create and maintain the visual, concept, narrative and auditory content used by the game.

Responsibilities:

- Create assigned visual, narrative and/or auditory assets according to the Issue requirements and acceptance criteria.
- Follow established art direction, technical constraints, naming conventions, and file organization.
- Coordinate with Developers when assets require implementation or technical integration.
- Submit art through the same branch and Pull Request workflow used by the rest of the team.
- Raise an Issue or communicate with the Lead Artist when requirements are unclear.

3.4. Lead Artist

The Lead Artist inherits all Artist responsibilities and coordinates visual work for the current milestone or Sprint.

Additional responsibilities:

- Review proposed art-related Issues before Producer approval.
- Clarify visual requirements and acceptance criteria.
- Coordinate work among Artists.
- Review art-related Pull Requests.
- Approve work that satisfies the Issue, art direction, and project standards.
- Request changes when submitted work is not ready for Producer approval.
- Communicate blockers and cross-discipline needs to the Producer and other Leads.

Lead assignments may change between milestones or Sprints.

3.5. Developer

Developers implement and maintain the technical parts of the game.

Responsibilities:

- Keep documentation up to date.
- Implement assigned gameplay, systems, scenes, tools, and fixes according to the Issue requirements.
- Work from the correct branch and avoid unapproved changes to shared branches.
- Test the behavior affected by an implementation or bug fix before opening a Pull Request.
- Coordinate with Artists and other disciplines when implementation depends on their work.
- Keep technical changes within the scope of the assigned Issue.

3.6. Lead Developer

The Lead Developer inherits all Developer responsibilities and coordinates technical work for the current milestone or Sprint.

Additional responsibilities:

- Review proposed implementation and bug Issues before Producer approval.
- Clarify technical requirements and acceptance criteria.
- Coordinate work among Developers.
- Review implementation and bug-fix Pull Requests.
- Approve work that satisfies the Issue and project standards.
- Request changes when submitted work requires correction before Producer approval.
- Help contributors when Git, Godot, or implementation problems block progress.

Lead assignments may change between milestones or Sprints.

4. Installing GitHub Desktop

TheArcade recommends **GitHub Desktop** for the standard team workflow. It provides a graphical interface for Git operations such as cloning, Fetching, Pulling, creating branches, committing, Pushing, and opening Pull Requests.

Use GitHub's official installation and setup documentation:

Installing GitHub Desktop

Getting started with GitHub Desktop

After installation:

1. Open GitHub Desktop.
2. Sign in using the GitHub account that belongs to TheArcade organization.
3. Confirm the name and email that will be attached to your commits.
4. Confirm that you can see the TheArcade repository when selecting a repository to clone.

For this workflow, contributors are **not required** to use command-line Git.

5. Installing Godot 4.7.2

The project uses **Godot 4.7.2 stable**.

Download the approved version from Godot's official archive:

Godot 4.7.2 stable download

Use the **Standard** (GDScript) build.



Do not upgrade the project to a different Godot version without explicit team approval. Opening or saving a project in a newer engine version can modify project files and create unnecessary incompatibilities for other collaborators.

After installation:

- Confirm that Godot 4.7.2 opens successfully.
- Keep the application in a stable location on your computer.
- Do not open the project using an unapproved engine version.

6. Cloning the Repository

Cloning creates a local copy of the shared GitHub repository on your computer.

Use GitHub's official GitHub Desktop cloning guide:

Cloning a repository from GitHub to GitHub Desktop

In general:

1. Sign in to GitHub and GitHub Desktop.
2. Open the TheArcade repository on GitHub.
3. Select **Code**.
4. Choose **Open with GitHub Desktop**.
5. Select a permanent folder on your computer for the repository.
6. Clone the repository.

Do not place the repository inside a temporary download folder or another location you are likely to delete accidentally.

6.1. Local and Remote Copies

It is important to understand that two related copies exist:

```
YOUR COMPUTER                          GITHUB

Local repository  <---- Pull ----  Remote repository
Local repository  ---- Push ---->  Remote repository
```

The **local repository** is the folder on your computer where you work.

The **remote repository** is the shared repository hosted on GitHub.

Saving a file changes your local files. It does **not** automatically update GitHub.

7. The Complete Contribution Workflow

The standard workflow is:

```
Discuss work
    |
    v
Create Issue
    |
    v
Lead approval
    |
    v
Producer approval
    |
    v
Product Backlog
```

```
    |
    v
Sprint begins
    |
    v
Issue assigned
    |
    v
Fetch -> Pull
    |
    v
Create branch
    |
    v
Work
    |
    v
Review changes
    |
    v
Commit -> Push
    |
    v
Create Pull Request
    |
    v
Lead review / approval
    |
    v
Producer review / approval
    |
    v
Producer merges
```

The following sections explain each stage.

7.1. 1. Discuss the Work

Before creating an Issue, discuss the idea, requirement, problem, or need with the appropriate people.

Discussion helps answer questions such as:

- Does this work actually need to be done?
- Is someone already working on it?
- Does it fit the current milestone or project direction?
- Which discipline owns it?
- Is it small enough to become one Issue?

- Does another task need to happen first?

Discussion does not replace an Issue. Once the work is sufficiently understood, it should be documented as an Issue before development begins.

7.2. 2. Create the Issue

The Issue is the formal description of a piece of work.

Every Issue should have:

7.2.1. Title

Use a short title that describes the desired result.

Good examples:

- `Create placeholder player idle sprite`
- `Implement main menu navigation`
- `Fix music restarting after scene transition`

Avoid vague titles such as `Art`, `Menu stuff`, or `Fix game`.

7.2.2. Label

Use the appropriate work label:

- **Asset Requirement** — a required art, audio, UI, animation, or other content asset.
- **Implementation** — new functionality or integration work.
- **Bug** — behavior that is incorrect or does not match the intended result.
- **Documentation** — create new or update existing

If you are unsure which label applies, ask the relevant Lead.

7.2.3. Objective

The objective states the result the Issue is trying to achieve.

Example:

```
Objective:
Add the supported desktop platforms to the project proposal.
```

7.2.4. Description

The description provides the context, requirements, limitations, references, or other information needed to understand the task.

Example:

```
Description:
The technical requirements section of the documentation does not currently
identify the desktop operating systems targeted by the project.
```

7.2.5. Acceptance Criteria

Acceptance criteria define what must be true for the Issue to be considered complete.

Example:

```
Acceptance Criteria:
- [ ] Windows is listed.
- [ ] macOS is listed.
- [ ] The information appears in the correct section.
- [ ] No unrelated documentation is changed.
```

Acceptance criteria should be observable whenever possible. They are the checklist used by the contributor and reviewer to determine whether the work is ready.

7.3. 3. Wait for Lead Approval

The appropriate Lead reviews the Issue before it enters the Product Backlog.

The Lead checks that:

- The task belongs to the relevant discipline.
- The requirements are understandable.
- The acceptance criteria are reasonable.
- The task is technically or creatively appropriate.
- The Issue is scoped well enough to be worked on.

The Lead may request clarification or changes to the Issue.

Do not begin implementation simply because the Issue exists.

7.4. 4. Wait for Producer Approval

After Lead approval, the Producer reviews the proposed work from a project-management perspective.

The Producer checks that:

- The work fits the current project scope.

- The work is worth scheduling relative to other priorities.
- Important dependencies are understood.
- The work is appropriate for the Product Backlog.

If approved, the Issue may be added to the Product Backlog.

This is the second approval gate before planned work enters the development queue.

7.5. 5. Add the Issue to the Product Backlog

The **Product Backlog** contains approved work that may be selected for future Sprints.

Being in the Product Backlog does **not** automatically mean that the Issue should be started immediately.

Backlog items wait until the team selects them for a Sprint according to priority, dependencies, available contributors, and current project goals.

7.6. 6. Sprint Start and Issue Assignment

At the beginning of a Sprint, the team selects a realistic set of approved backlog items to pursue.

An Issue should be assigned to the collaborator responsible for completing it before work begins.

Before starting an assigned Issue, confirm:

- The Issue is part of the current Sprint or has otherwise been authorized for immediate work.
- You are the assigned collaborator.
- You understand the objective.
- You understand every acceptance criterion.
- You know which branch should be used as the starting point.

If any requirement is unclear, ask before editing project files.

7.7. 7. Fetch

Open GitHub Desktop and select the TheArcade repository.

First verify:

- You are looking at the correct repository.
- You are on the branch from which your Issue branch should be created, usually `main` or another branch specified by a Lead.

Use **Fetch origin**.

Fetch checks GitHub for new remote commits and updates your local knowledge of the remote

repository. Fetch does not by itself apply those changes to the files in your current working branch.

Use Fetch before beginning work so that GitHub Desktop can tell you whether your local branch is behind the remote branch.

7.8. 8. Pull

If GitHub Desktop reports that the current branch has remote changes available, use **Pull origin** before creating your Issue branch.

Pull brings the latest remote commits into your local copy of the current branch.

The goal is simple: **create your new branch from the most current approved version of its parent branch**.

If Pull reports a conflict or anything unexpected, stop and ask a Lead.

7.9. 9. Create the Branch

Never perform normal Issue work directly on `main`.

Create a separate branch for the assigned Issue from the correct parent branch.

Recommended naming pattern:

```
asset/<issue-number>-short-description
impl/<issue-number>-short-description
bug/<issue-number>-short-description
doc/<issue-number>-short-description
```

Examples:

```
asset/23-player-idle
impl/41-main-menu
bug/57-audio-loop
```

Before editing anything, visually confirm the **Current Branch** shown by GitHub Desktop.

A useful habit is:

```
Correct repository?
Correct branch?
Then work.
```

7.10. 10. Work on the Branch

Use the appropriate project tool to complete the assigned task. Depending on your role, this may be Godot, an art application, an audio application, or a text editor.

While working:

- Stay within the scope of the assigned Issue.
- Check the acceptance criteria as you work.
- Avoid modifying unrelated files.
- Test or inspect your work as appropriate.
- Save files normally.

If you discover unrelated work that should be done, do not quietly include it in the current branch. Discuss it and create a separate Issue when appropriate.

7.11. 11. Review Changes Before Committing

Return to GitHub Desktop and inspect the **Changes** panel.

Ask yourself:

- Are these the files I intended to change?
- Did I accidentally modify something unrelated?
- Does the work match the Issue?
- Does it satisfy the acceptance criteria?
- Are temporary or generated files appearing unexpectedly?

Do not commit files you do not understand.

7.12. 12. Commit

A **commit** is a named checkpoint in the local history of your branch.

A commit is **not** the same as saving a file and is **not** the same as uploading work to GitHub.

Use a short, meaningful commit message that describes what changed.

Recommended style:

```
Verb + what changed
```

Good examples:

```
Add supported platforms to proposal
```

```
Create placeholder player idle sprite
Implement main menu navigation
Fix music restart after scene change
```

Poor examples:

```
stuff
update
final
final-final-2
asdf
```

For larger Issues, multiple sensible commits are acceptable. Each commit should represent a useful checkpoint rather than an arbitrary save.

7.13. 13. Push

A commit remains on your computer until it is pushed.

Push sends your local commits to the corresponding branch on GitHub.

Remember:

```
Save file != Commit
Commit != Push
```

After Pushing, the branch and its commits are available on GitHub for the team to inspect.

If GitHub Desktop refuses the Push because the remote branch has changed, do not force the operation. Fetch and ask a Lead if you are unsure how to proceed.

7.14. 14. Create the Pull Request

When the assigned work satisfies the Issue and acceptance criteria, create a **Pull Request** (PR).

A Pull Request means:

```
My work is ready to be reviewed before it is added to the target branch.
```

Before submitting the Pull Request:

- Confirm the correct source branch.
- Confirm the correct target/base branch.
- Use a clear Pull Request title.
- Summarize what changed.

- Reference the related Issue.
- Confirm the acceptance criteria are satisfied.
- Request review from the appropriate Lead.

Do not merge your own Pull Request unless your role and the project's current policy explicitly authorize it.

7.15. 15. Lead Review and Approval

The appropriate Lead reviews the Pull Request first.

The Lead checks that:

- The submitted work matches the Issue.
- Acceptance criteria are satisfied.
- The change follows discipline-specific standards.
- No obvious unrelated changes are included.
- The contribution is ready to move to Producer review.

The Lead may either approve the Pull Request or request changes.

7.15.1. If Changes Are Requested

Do **not** create a new branch or a new Pull Request.

Stay on the same Issue branch:

```
Make requested change
        |
        v
Review files
        |
        v
Commit
        |
        v
Push
        |
        v
Existing Pull Request updates automatically
        |
        v
Request / await review again
```

Review feedback is a normal part of collaboration.

7.16. 16. Producer Review and Approval

After Lead approval, the Producer performs the final project-level review.

The Producer checks that the contribution is appropriate to integrate into the shared target branch and that the required review process has been completed.

The Producer may:

- Approve the Pull Request for merge.
- Request additional clarification or changes.
- Delay the merge if another dependency or project concern must be addressed first.

7.17. 17. Producer Merge

Once the Pull Request has the required approvals, the Producer merges it into the target branch.

After the merge:

- The contribution becomes part of the shared project history.
- The related Issue can be closed when the acceptance criteria have been satisfied.
- Other collaborators can Fetch and Pull the newly merged work.
- The Issue branch should no longer be used for unrelated work.

A new task should normally begin with a new Issue and a new branch.

8. Workflow Summary

```
1.  Discuss proposed work
2.  Create Issue
3.  Lead reviews / approves Issue
4.  Producer reviews / approves Issue
5.  Issue enters Product Backlog
6.  Sprint begins
7.  Issue is selected and assigned
8.  Select correct parent branch
9.  Fetch origin
10. Pull origin if updates are available
11. Create Issue branch
12. Complete the work
13. Review changed files
14. Commit
15. Push
16. Create Pull Request
17. Lead reviews / approves PR
18. Producer reviews / approves PR
19. Producer merges
```

```
20. Issue is completed / closed
```

9. Git and GitHub Vocabulary

9.1. Repository

The collection of project files together with their Git history.

9.2. Local Repository

The copy of the repository stored on your computer.

9.3. Remote Repository

The shared repository hosted on GitHub.

9.4. Issue

A documented unit of work containing the objective, description, acceptance criteria, and relevant project information.

9.5. Product Backlog

The ordered collection of approved work that may be selected for future Sprints.

9.6. Sprint

A short development cycle during which the team works toward a defined set of goals using selected backlog items.

9.7. Branch

An isolated line of work used to complete an Issue without directly editing the shared target branch.

9.8. Fetch

Checks the remote repository and updates your local Git information about remote changes without applying those changes to your working files.

9.9. Pull

Brings remote commits into your current local branch.

9.10. Commit

A named checkpoint containing selected changes in the local Git history.

9.11. Push

Sends local commits to GitHub.

9.12. Pull Request

A request to review and eventually integrate one branch into another.

9.13. Review

The process in which a Lead or other authorized reviewer examines a Pull Request and either approves it or requests changes.

9.14. Merge

The act of integrating approved work from one branch into another. Under the standard TheArcade workflow, the Producer performs the final merge after the required approvals.

10. TheArcade Golden Rules

1. Every planned contribution begins with discussion and an Issue.
2. Every Issue must have an objective, description, and acceptance criteria.
3. Use the correct Issue label.
4. Do not begin work until the required approvals have been completed and the Issue has been assigned for work.
5. Never perform normal project work directly on `main`.
6. Fetch and Pull the correct parent branch before creating your Issue branch.
7. Check the repository and current branch before editing files.
8. Keep each branch focused on its Issue.
9. Review changed files before every commit.
10. Use meaningful commit messages.
11. Push commits so the team can review them.
12. Use a Pull Request to propose integration into the target branch.
13. The appropriate Lead reviews first.
14. The Producer gives final approval and performs the merge.
15. If changes are requested, update the same branch and Push again; do not create a replacement

Pull Request.

16. If you encounter a merge conflict or unfamiliar Git state, stop and ask for help.

11. What You Are Not Expected to Know Yet

New collaborators are not expected to independently perform advanced Git operations such as:

- Resolving merge conflicts.
- Rebasing.
- Force Pushing.
- Rewriting Git history.
- Cherry-picking commits.
- Reverting other contributors' work.
- Recovering lost commits.
- Using command-line Git.
- Choosing complex merge strategies.

Those operations may become useful later, particularly for Leads and Developers, but they are not required to follow the standard contribution workflow.

12. When to Ask for Help

Stop and contact a Lead or Producer when:

- GitHub Desktop reports a merge conflict.
- You accidentally worked on `main`.
- You created work from the wrong branch.
- A Push or Pull fails and you do not understand the reason.
- You see changed files that you did not intentionally modify.
- You are unsure whether a file should be committed.
- You believe the Issue requirements need to change significantly.
- Your work depends on another unfinished Issue.
- You are unsure which branch should be the Pull Request target.
- You think you may have lost work.

Do not experiment with destructive Git actions on the shared project simply to see whether they solve the problem. Asking early is usually much faster than repairing damaged history later.

13. Quick Pre-Work Checklist

Before beginning an assigned Issue:

```
[ ] The Issue is approved.
[ ] The Issue is authorized for the current Sprint or immediate work.
[ ] I am assigned to it.
[ ] I understand the objective.
[ ] I understand the acceptance criteria.
[ ] GitHub Desktop is on the correct repository.
[ ] I selected the correct parent branch.
[ ] I fetched remote changes.
[ ] I pulled any available updates.
[ ] I created / selected the correct Issue branch.
[ ] I verified the current branch before editing.
```

Before opening a Pull Request:

```
[ ] The Issue requirements are complete.
[ ] The acceptance criteria are satisfied.
[ ] I reviewed all changed files.
[ ] My commits have meaningful messages.
[ ] I pushed the latest commits.
[ ] The Pull Request targets the correct branch.
[ ] The Pull Request references the Issue.
[ ] I requested review from the appropriate Lead.
```

14. Useful Official Guides

- Create a GitHub account
- GitHub organization invitations
- Install GitHub Desktop
- Get started with GitHub Desktop
- Clone a repository with GitHub Desktop
- Download Godot 4.7.2 stable

15. Final Reminder

Pushing **directly** to **main** is a crime.

The standard contribution loop is:

```
Discuss -> Issue -> Approvals -> Backlog -> Sprint -> Assignment
```

```
        -> Fetch -> Pull -> Branch -> Work -> Commit -> Push
        -> Pull Request -> Lead Approval -> Producer Approval -> Merge
```

You do not need to memorize every Git feature. You need to follow this workflow consistently, keep your work scoped to the assigned Issue, and ask for help when the safe path is unclear.